ME315 - Laboratório 11

Alejandro Francisco Figueiredo 230532

Questão 1

```
import polars as pl
import pandas as pd
from plotnine import *
from plotnine.data import diamonds

# Base de dados usada no laboratório
# plotnine fornece o conjunto diamonds com 53940 linhas
df = pl.DataFrame({col: diamonds[col].astype(str).tolist() if str(diamonds[col].dtype) == 'category' else
print(df.shape)
df.head(5)
```

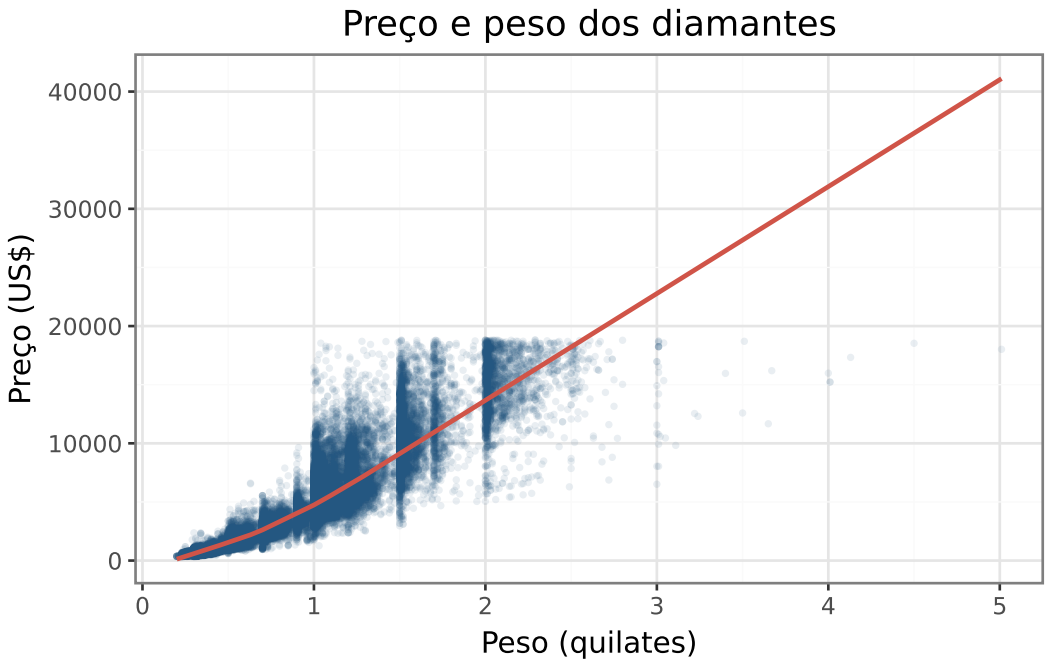
```
(53940, 10)
```

	carat f64	cut str	color str	clarity str	depth f64	table f64	price i64	x f64	y f64	z f64
0.23		”Ideal”	”E”	”SI2”	61.5	55.0	326	3.95	3.98	2.43
0.21		”Premium”	”E”	”SI1”	59.8	61.0	326	3.89	3.84	2.31
0.23		”Good”	”E”	”VS1”	56.9	65.0	327	4.05	4.07	2.31
0.29		”Premium”	”I”	”VS2”	62.4	58.0	334	4.2	4.23	2.63
0.31		”Good”	”J”	”SI2”	63.3	58.0	335	4.34	4.35	2.75

Questão 2

```
df_grafico = df.select('carat', 'price').to_pandas()

grafico1 = (
    ggplot(df_grafico, aes(x='carat', y='price'))
    + geom_point(alpha=0.10, size=0.55, color='#245781')
    + geom_smooth(method='lowess', se=False, color='#d05448')
    + labs(title='Preço e peso dos diamantes', x='Peso (quilates)', y='Preço (US$)')
    + theme_bw()
)
grafico1
```

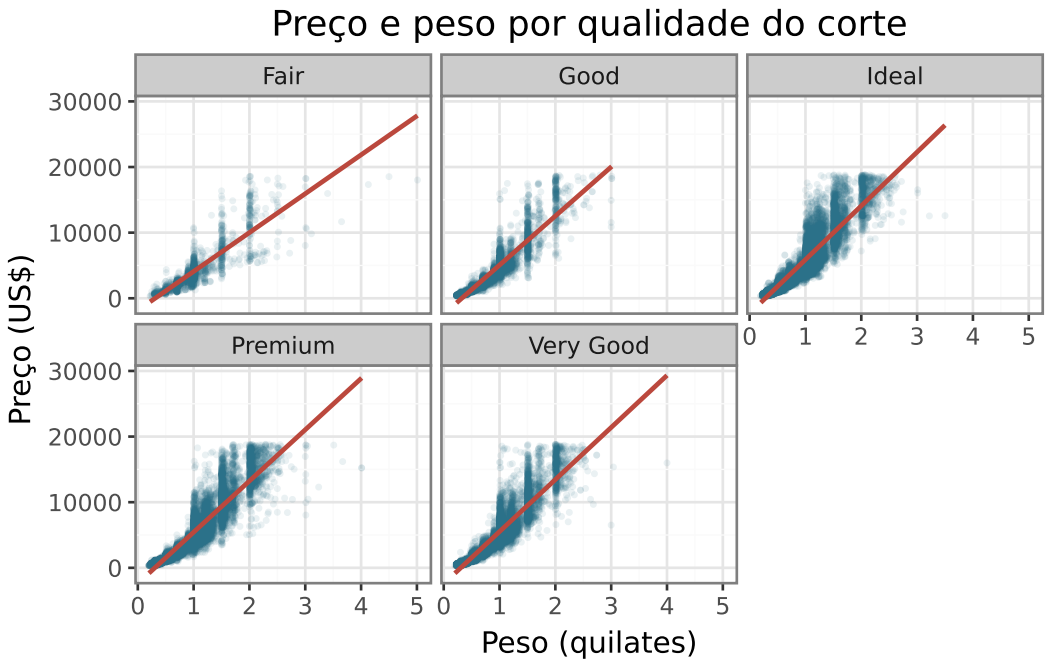


O gráfico mostra que diamantes mais pesados geralmente têm preços maiores. Essa relação não parece ser uma reta, porque o preço cresce mais rapidamente para valores altos de quilates. Usei pontos com transparência para diminuir a sobreposição, já que são muitos diamantes, e uma curva para facilitar a visualização da tendência. Também coloquei título e nomes nos eixos para deixar o gráfico mais claro.

Questão 3

```
df_corte = df.select('carat', 'price', 'cut').to_pandas()

grafico2 = (
    ggplot(df_corte, aes(x='carat', y='price'))
    + geom_point(alpha=0.10, size=0.45, color='#2b7189')
    + geom_smooth(method='lm', se=False, color='#bb483d')
    + facet_wrap('~cut', ncol=3)
    + labs(title='Preço e peso por qualidade do corte', x='Peso (quilates)', y='Preço (US$)')
    + theme_bw()
)
grafico2
```



Separando pelo tipo de corte, ainda podemos observar que o preço tende a aumentar com o peso. Porém, os grupos não apresentam exatamente o mesmo comportamento: há diferenças na dispersão dos valores e também nas inclinações das linhas. A divisão em painéis ajuda a comparar os cortes sem colocar todos os pontos no mesmo gráfico. Isso não significa que o corte sozinho explica o preço, porque cor e clareza também podem influenciar.

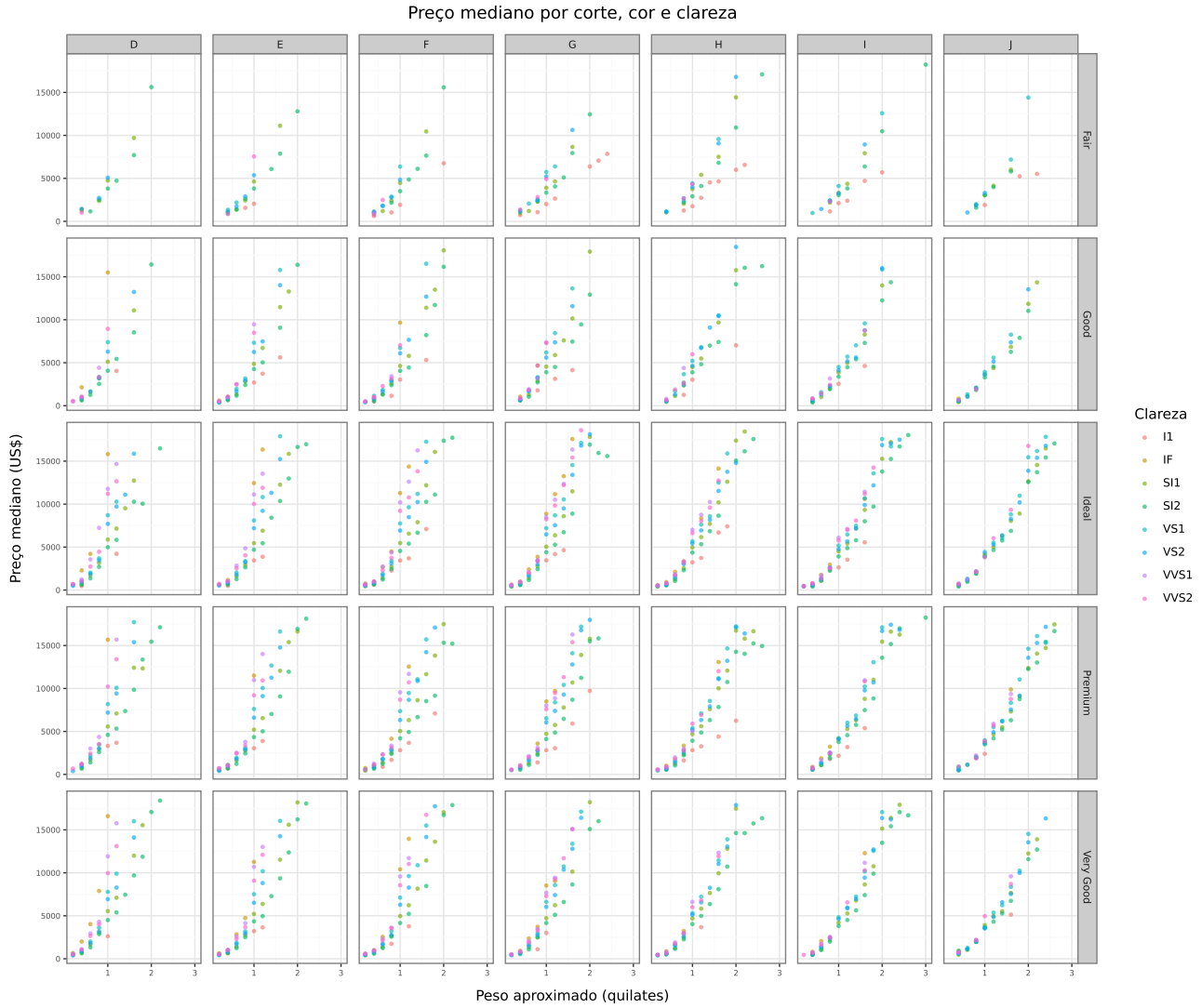
Questão 4

```
df_resumo = (
    df
    .with_columns((pl.col('carat') / 0.2).round(0).mul(0.2).alias('faixa_peso'))
    .group_by(['cut', 'color', 'clarity', 'faixa_peso'])
    .agg(
        pl.col('price').median().alias('preco_mediano'),
        pl.len().alias('n')
    )
    .filter(pl.col('n') >= 3)
    .sort(['cut', 'color', 'clarity', 'faixa_peso'])
)

df_resumo.head(5)
```

cut str	color str	clarity str	faixa_peso f64	preco_mediano f64	n u32
”Fair”	”D”	”IF”	0.4	1440.0	3
”Fair”	”D”	”SI1”	0.4	1253.0	7
”Fair”	”D”	”SI1”	0.8	2376.0	21
”Fair”	”D”	”SI1”	1.0	4751.0	23
”Fair”	”D”	”SI1”	1.6	9704.0	3

```
grafico3 = (
    ggplot(df_resumo.to_pandas(), aes(x='faixa_peso', y='preco_mediano', color='clarity'))
    + geom_point(alpha=0.65, size=0.9)
    + facet_grid('cut~color')
    + labs(title='Preço mediano por corte, cor e clareza',
           x='Peso aproximado (quilates)', y='Preço mediano (US$)', color='Clareza')
    + theme_bw()
    + theme(figure_size=(13, 11),
            axis_text_x=element_text(size=6),
            axis_text_y=element_text(size=6),
            strip_text=element_text(size=8))
)
grafico3
```



A principal dificuldade foi representar as três variáveis categóricas junto com peso e preço: são muitas combinações e um gráfico só ficaria muito carregado. Por isso, usei painéis para corte e cor, cores diferentes para os níveis de clareza e o preço mediano em faixas de peso de 0,2 quilate. Também retirei grupos com menos de três observações, para evitar dar muita importância a combinações muito raras. Em geral, o preço aumenta com o peso nas diferentes configurações, mas não da mesma forma em todos os grupos. É possível notar variação de preços para pesos parecidos, mostrando que as demais características também importam.

```python
from datetime import datetime

agora = datetime.now()
print(agora.strftime("%H:%M:%S"))
```

```
21:17:08
```